

Active Noise Cancellation (ANC) For Airplane Engine Noise

1 Background & Objective

This project takes a deep dive into Active Noise Cancellation (ANC). While we see this technology every day in consumer devices like AirPods and Bose headphones, it was originally developed out of necessity for aviation. In the 1950s, engineers like Dr. Lawrence Fogel designed early, bulky ANC systems specifically to protect pilots' hearing and allow them to communicate over deafening helicopter and airplane engines. The technology has come a long way since then, but for this project, we are returning to that original motivating use case.

Engine noise is predictable to a meaningful extent because it contains persistent tonal and broadband structures tied to engine rotation, harmonics, and airframe acoustics. As DSP Engineers, this should excite you. The more structure you have in your signals, the more assumptions you can make, the better your algorithms are. In this challenge, you will need to analyze and exploit the structure of the engine noise in order to create a working active noise canceler.

To do this successfully, you have to understand the fundamental mechanism at play. ANC uses the statistical properties of a signal and real-time destructive interference to constantly track and counter dynamic acoustic environments. In a standard feedforward topology, a reference microphone captures the incoming ambient noise, $x(n)$. A DSP then uses an adaptive digital filter with weights $\mathbf{w}(n)$ to generate an anti-noise signal, $y(n) = \mathbf{w}^T(n)\mathbf{x}(n)$. This is played through a speaker to cancel the original primary noise, $d(n)$, at the listener's ear. For perfect cancellation, the acoustic anti-noise arriving at the ear, $y'(n)$, must be the exact inverse of the primary noise ($y'(n) = -d(n)$), ensuring the residual sound monitored by the error microphone approaches zero: $e(n) = d(n) + y'(n) \approx 0$.

The physical journey of the anti-noise from the speaker to the error microphone introduces inherent phase shifts and delays, which presents a significant challenge in solving this problem.

For this problem, you are given a simulator that models these exact acoustic paths between a noise source, microphones, and a speaker. Your task is to design and implement an anti-noise control algorithm that minimizes the residual sound measured at the in-ear microphone.

You will implement your algorithm in `src/anc.cpp` via the following callback:

```
void anc::step(const MicBlock &micBlock, dsp::Block &control);
```

For each incoming audio block, the simulator provides microphone data in `micBlock`. You must populate `control` with the anti-noise block to be played through the speaker.

2 I/O Interface and Data Structures

Each callback provides a `MicBlock` struct:

```
struct MicBlock {
    dsp::Block rawInput;
    dsp::Block outside;
    dsp::Block inear;
    std::chrono::time_point timestamp;
    uint64_t seq;
};
```

In this simulator, we are block processing. This means that our hardware is reading in the signal one block at a time. We do this to give our system enough time to compute the necessary computations. For example, if we try to compute a 4096 point FFT every sample, at a 48kHz sampling rate, that gives us around 20 microseconds to compute the FFT. This is not reasonable for any embedded device.

For this reason, we buffer up our signal to 256 samples. At 48 kHz, that gives us around 5.3 ms for each computation, which is reasonable. The main downside of this is now we have an extra 5.3 ms of latency. This makes broadband ANC near impossible. But, since we have structure in our engine noise, ANC is still very possible to implement in our scenario.

Now let's get to the C++ of it all. The `dsp::Block` is our primary data container. Under the hood, it is defined as:

```
using Block = Eigen::Matrix<float, 256, 1>;
```

`Block` is a fixed-size column vector of 256 single-precision floats. It is stored as a contiguous memory buffer that is allocated strictly on the stack, meaning we avoid unpredictable heap allocations that may cause audio dropouts. Because the block size is known at compile time, Eigen can aggressively optimize operations if you set the right compiler flags to optimize the code. *(Note: Assume you must set the proper compiler flags. I am not giving you the correct compiler flags to use. As an engineer, it is up to you to enable this and learn how to write and build fast code. This is a critical skill for industry).* Additionally, strict timing is critical in real-time acoustic systems.

- `rawInput`: The raw WAV block being fed into the simulation.
- `outside`: The simulated outside (reference) microphone block.
- `inear`: The simulated in-ear (error) microphone block.
- `timestamp`: The high-resolution time the block was captured, utilized for profiling.
- `seq`: The block sequence number.

Your ANC algorithm must output `control`, the speaker command block that the simulator uses to generate the anti-noise signal.

3 System Timing and Constraints

The simulator operates at a fixed sample rate of 48,000 Hz with a block size of 256 samples. This means each block represents exactly $256/48000$, or roughly 5.33 ms of audio.

3.1 The Control Loop

The simulator uses block-by-block processing, but the control signal you compute is not applied to the speaker immediately within that same block. In the default setup, the system introduces one full block of transport delay before the control block reaches the speaker. This adds about 5.33 ms of explicit buffering latency, on top of the physical delay in the acoustic plant.

Because the control signal computed in block k doesn't actuate the speaker until block $k + 1$, your controller must be causal and delay-aware from the start.

3.2 Acoustic and Transport Latency

The primary delays you must account for are:

1. The explicit one-block software transport delay.
2. Physical acoustic propagation times.
3. Frequency-dependent phase shifts inherent to the modeled acoustic paths (P, S, C, and H).

These constraints strictly dictate system performance. Increased acoustic delay narrows your margin of error for matching the phase of the anti-noise to the disturbance. Because high-frequency cancellation is highly sensitive to tiny timing mismatches, tracking and canceling high frequencies is significantly harder than low frequencies. Even at low frequencies, the physical paths alter the gain and phase relationship between the noise source and your actuator.

4 The Latency Problem

ANC is fundamentally a latency-sensitive problem.

To successfully reduce noise, your anti-noise must hit the in-ear microphone with the correct timing, sign, and phase. If your controller reacts too late, the speaker output will miss the disturbance entirely—or worse, reinforce it.

This is especially critical because the most objectionable components of cockpit noise often live in low-frequency bands and engine harmonics. These repeatable components are exactly what you want to exploit, but you can only cancel them if your controller aligns with them perfectly in time and phase.

You are balancing several sources of latency pressure:

- The controller only updates once per 256-sample block.

- The simulator adds a mandatory 1-block transport delay.
- The disturbance and control signals travel through entirely different acoustic paths.
- Controller state, smoothing, prediction, or filtering inherently introduces algorithm delay.

Thus, more aggressive filtering makes your control signal smoother, but too much computational lag means the anti-noise arrives with the wrong phase. Keep this balance in mind during development.

5 Real-Time Constraints and Optimization

Your controller sits directly in a time-critical audio path. You must carefully distinguish between RT-safe (Real-Time safe) and RT-unsafe code.

RT-unsafe operations—like heap allocations, file I/O, console printing (`std::cout`), sleeping, or blocking locks—can introduce timing jitter. As a rule of thumb: do all your setup and file loading before processing starts, keep the per-block math bounded, and avoid dynamically resizing arrays (e.g., growing a `std::vector`) inside the processing loop.

In production ready DSP systems, it is often common practice to annotate your functions with:

```
// RT-safe
float doSafeStuff(int i) {
    // code here
    int p = i+3;
    std::array<int, 2> xyz = {i, p};
    return 0.3f;
}
```

or

```
// RT-unsafe
void doUnsafeStuff() {
    // heap allocation here
    std::vector<int> abc{};
    for(size_t i = 0; i < 1_000_000; ++i) {
        abc.push_back(i);
    }
}
```

If your code in `anc::step(...)` fails to complete within its allotted 5.33 ms deadline, the system defaults your control signal to zeros, resulting in a dropped audio frame.

5.1 CPU Profiling & Environment Setup

If your algorithm gets bogged down, profile it to figure out where the CPU time is going. This will tell you if a helper routine is too expensive, or if your controller scales poorly when you add filter states.

5.1.1 Windows / Linux: WSL2 and perf

Windows does not natively support `perf`, but you can replicate the exact Linux profiling workflow by utilizing the Windows Subsystem for Linux (WSL2). First, install WSL from an Administrator PowerShell prompt:

```
wsl --install
```

Once initialized, open your WSL terminal and update dependencies to include the C++ toolchain, `perf`, and the FFTW library:

```
sudo apt update
sudo apt install build-essential cmake linux-tools-generic linux-tools-common libfftw3-dev
```

Run your profiling trial using the tools below:

```
sudo perf record -g -- ./build/src/DSPApp input.wav profile_run
```

```
# To view an interactive CLI report
sudo perf report
```

```
# To export data for a flamegraph visualization
sudo perf script | stackcollapse-perf.pl | flamegraph.pl > flamegraph.svg
```

Open `flamegraph.svg` in a browser. The widest stacks show where the most time is being spent.

5.1.2 macOS: Clang, Homebrew, and Instruments

To compile and profile natively on macOS, install the build tools via Terminal:

```
xcode-select --install
```

Set up Homebrew and install `cmake` along with `FFTW`:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew update
brew install cmake fftw
```

Profile your binary natively using Instruments via `xctrace`:

```
xctrace record --template 'Time Profiler' --output anc_profile.trace --
  launch -- ./build/src/DSPApp input.wav profile_run
open anc_profile.trace
```

6 Plant Architecture (Acoustic Paths)

We can model the different acoustic paths in the plant using LTI systems.

(Note: The simulator is not perfectly static. Some acoustic paths drift slightly over time to simulate the reality that earbuds and headsets shift. While you can treat the paths as LTI over short windows, expect small fluctuations over time—just like a pilot shifting their head or adjusting their helmet).

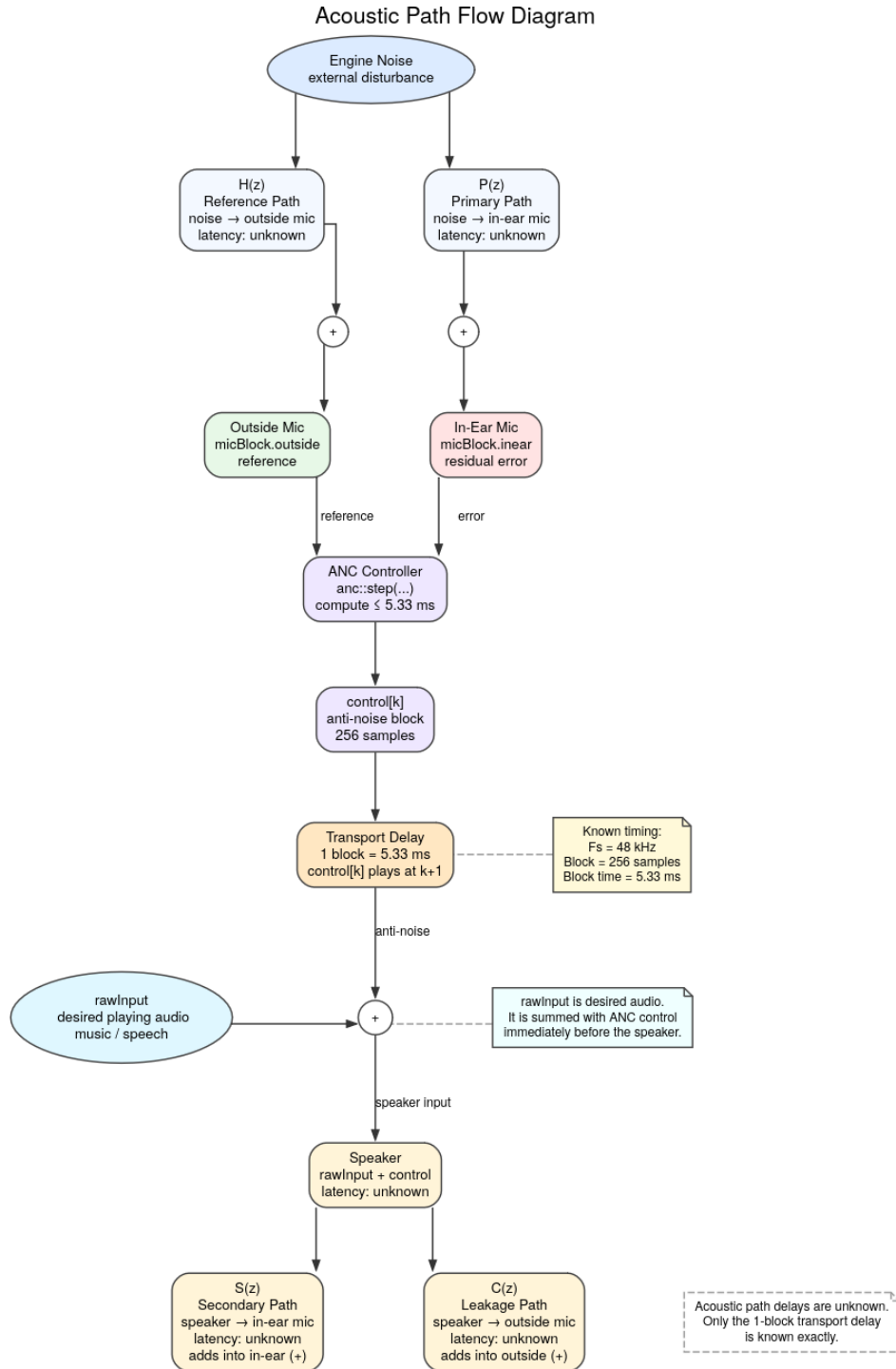


Figure 1: ANC System Block Diagram

6.1 Path Definitions

Reference Path (H)

Maps the disturbance to the outside microphone. This provides our “noise” measurement.

Primary Path (P)

Maps the disturbance to the in-ear microphone. This is the noise you are actively trying to cancel.

Secondary Path (S)

Maps your speaker command to the in-ear microphone. This is critical: your **control** block is reshaped by the speaker, enclosure, and ear acoustics before it reaches the **inear** mic.

Feedback/Leakage Path (C)

Maps the speaker command back into the outside microphone. Your reference mic is not perfectly isolated from your speaker output.

Speaker Path

Speakers are not perfect actuators. They have their own system response.

6.2 Causal Chains

While the simulator’s exact delays are hidden, the chain of when a given signal arrives is known.

Disturbance Path:

noise $\rightarrow$ H $\rightarrow$ **outside** mic

noise $\rightarrow$ P $\rightarrow$ **inear** mic

Control Path:

control signal $\rightarrow$ buffered for 1 block $\rightarrow$ played by speaker $\rightarrow$ S $\rightarrow$ **inear** mic.

control signal $\rightarrow$ buffered for 1 block $\rightarrow$ played by speaker $\rightarrow$ C $\rightarrow$ **outside** mic.

Because these paths don’t line up perfectly in time, phase-sensitive cancellation is difficult. It is only possible when the reference signal provides enough advance warning to overcome the total control-path delay.

7 Design Considerations

Design an algorithm that uses the available signals to reduce the residual energy at the in-ear microphone.

As you design your architecture, you must address the following engineering problems:

- Which microphone signal should drive the controller: `outside`, `inear`, or both?
- How will past blocks influence the current control block?
- How will you account for the known 1-block actuation delay?
- How will your system remain stable when the anti-noise leaks back into the outside reference mic?
- How can you exploit the fact that engine/wind noise has an identifiable spectral structure?

You have complete freedom over the control strategy, state management, and DSP structure. The success criterion is simply producing a `control` block that measurably reduces the noise amplitude at `inear`.

8 Implementation and Dependencies

You will primarily work in `src/anc.cpp` and `include/anc.h`, however, you are allowed and encouraged to create supplementary classes, files and code. Any file under `src/` is fair game to modify.

For linear algebra, Eigen is the required matrix/vector library for this project. If you need frequency-domain processing, you may use Eigen's built-in FFT support, or explicitly use an FFT library like FFTW.

9 Offline Prototyping (Python)

You are strongly encouraged to prototype your algorithm in Python before writing any C++. Python is much better suited for quick iteration, plotting, and offline debugging.

I suggest the following workflow:

1. Run the C++ simulator with an empty controller and export the mic WAV files.
2. Read those WAV files into Python.
3. Prototype and tune your algorithm offline.
4. Port the working block logic to `src/anc.cpp`.

```
from scipy.io import wavfile
import numpy as np

# Read
sample_rate, x = wavfile.read("sample.wav")
assert sample_rate == 48000
x = x.astype(np.float32) / 32768.0

# Write
y = np.clip(y, -1.0, 1.0)
wavfile.write("python_output.wav", 48000, (y * 32767.0).astype(np.int16))
```


9.1 Reference Block Framework

The simulator processes 256-sample blocks. Your Python script should mimic this constraint. Here is a simple (and very poor) static feedforward baseline to demonstrate block iteration:

```
from scipy.io import wavfile
import numpy as np

BLOCK_SIZE = 256
sample_rate, x = wavfile.read("sample.wav")
x = x.astype(np.float32) / 32768.0
y = np.zeros_like(x)

for start in range(0, len(x) - BLOCK_SIZE + 1, BLOCK_SIZE):
    block = x[start:start + BLOCK_SIZE]

    # Write your control logic here

    y[start:start + BLOCK_SIZE] = control

wavfile.write("out.wav", 48000, np.clip(y * 32767.0, -32768, 32767).astype(np.int16))
```

Once your Python prototype works, port the logic to C++, maintaining exact block-by-block state management, sign conventions, and scaling.

10 Build and Run Instructions

Compile the simulation target from your environment root repository:

```
cmake -S . -B build
cmake --build build --target DSPApp
```

Run the simulator execution binary:

```
./build/src/DSPApp src/input.wav trial1
```

This generates output files (`trial1_raw_input.wav`, `trial1_outside_mic.wav`, `trial1_inear_mic.wav`) that you can inspect in audio programs such as Audacity (<https://www.audacityteam.org/>).

11 Generating Test Signals

Your test files should be standard uncompressed WAV files: 48 kHz, mono channel, 16-bit PCM. You can easily generate test files in Audacity:

1. Set the project rate to 48000 Hz.
2. Generate or record a signal (impulses, white noise, tones, etc.).
3. Mix down to Mono if necessary.
4. Export as WAV (Microsoft) → Signed 16-bit PCM.

We highly recommend testing your controller against impulses, multi-tone mixtures, and slowly varying broadband noise to evaluate robustness across different environments. You can generate these test signals in Audacity or using Python.

12 Grading Rubric

Your grade is based on three components:

12.1 Technical Report (70%)

This is the core of your grade. Your writeup must explain your overall approach, assumptions, what you tried (and why), what failed, and the limitations of your final design. You must include plots (spectra, error metrics, etc.) and comparisons between approaches. We care most about your thought process and understanding, not if the code works perfectly.

12.2 Code Architecture (10%)

Your `anc.cpp` implementation should be clean, readable, and well-organized. Real-time constraints must be respected (no heap allocations or blocking I/O in the callback path).

12.3 Algorithm Effectiveness (20%)

Your controller should meaningfully reduce residual noise at the in-ear microphone. Evaluate your framework utilizing the following multi-metric analysis:

- **Noise RMS reduction:** Look at how the power of noise shrinks due to your algorithms.
- **Low-frequency band-power reduction:** Crucial for engine noise.
- **Spectrograms:** Excellent for spotting narrowband reduction, transient failures, or accidental amplification (howling).
- **PESQ or STOI:** Useful for evaluating intelligibility if the input contains speech (a common scenario for pilot comms). Use `scriptscompare_wav.py`.

Do not optimize blindly for a single metric. A controller that achieves a high RMS score but randomly rings or deafens the user with feedback is not valuable. Use the metrics alongside listening tests to understand your system's true behavior.

13 Submission Requirements

Submit a link to your GitHub repository containing:

1. Your `anc.cpp` implementation.

2. Your supporting Python code and plots.
3. Your technical report.